THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

Socket Programming and Intro to application protocols

Dr Lachlan Andrew

© University of Melbourne 2025

Recap

- Protocols are modules to solve different network problems
- Protocol stack

© 2025 University of Melbourne

Summary

- High level overview of socket programming
  - Relationship to
    - application layer (e.g., HTTP)
    - transport layer (e.g., TCP)
  - Sockets in C

© 2025 University of Melbourne

Hello, network! (C “Client” socket)

```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int connfd = 0, re = 1, s, n;
struct addrinfo hints, *res, *rp;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET6;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_ACTVE;
```

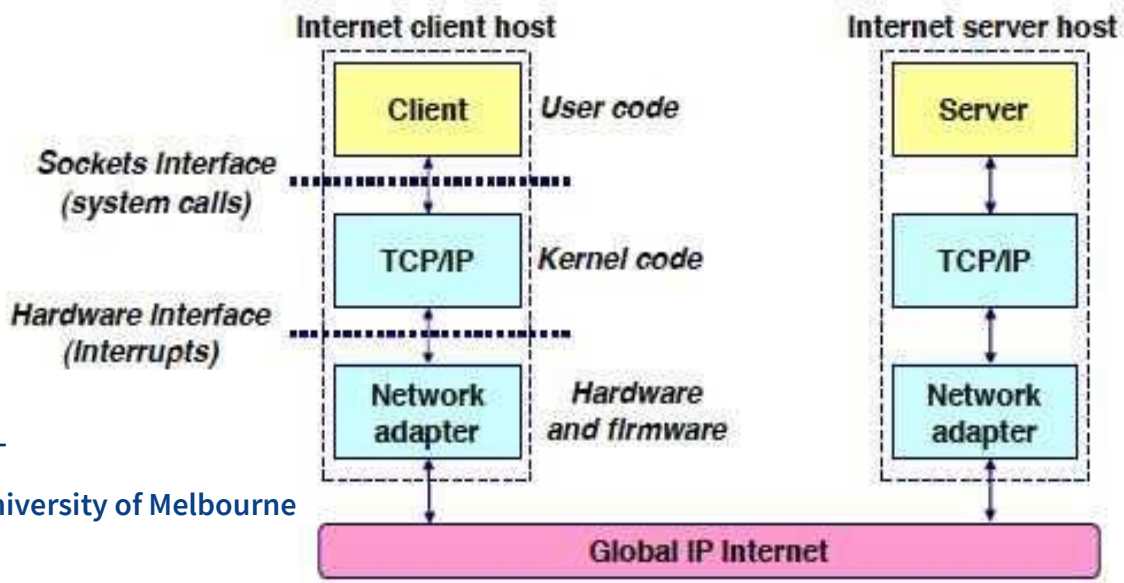
```
s = getaddrinfo(“localhost",
                “5000”, &hints, &res);
for (rp = res; rp != NULL;
            rp = rp->ai_next) {
    connfd = socket(rp->ai_family,
                    rp->ai_socktype,
                    rp->ai_protocol);
    if (connfd == -1) continue;
    if (connect(connfd,
                rp->ai_addr,
                rp->ai_addrlen) != -1
        break;
    close(connfd);
}
write (connfd, “Hello, network!\r\n”);
close (connfd);
```

© 2025 University of Melbourne

What is a socket?

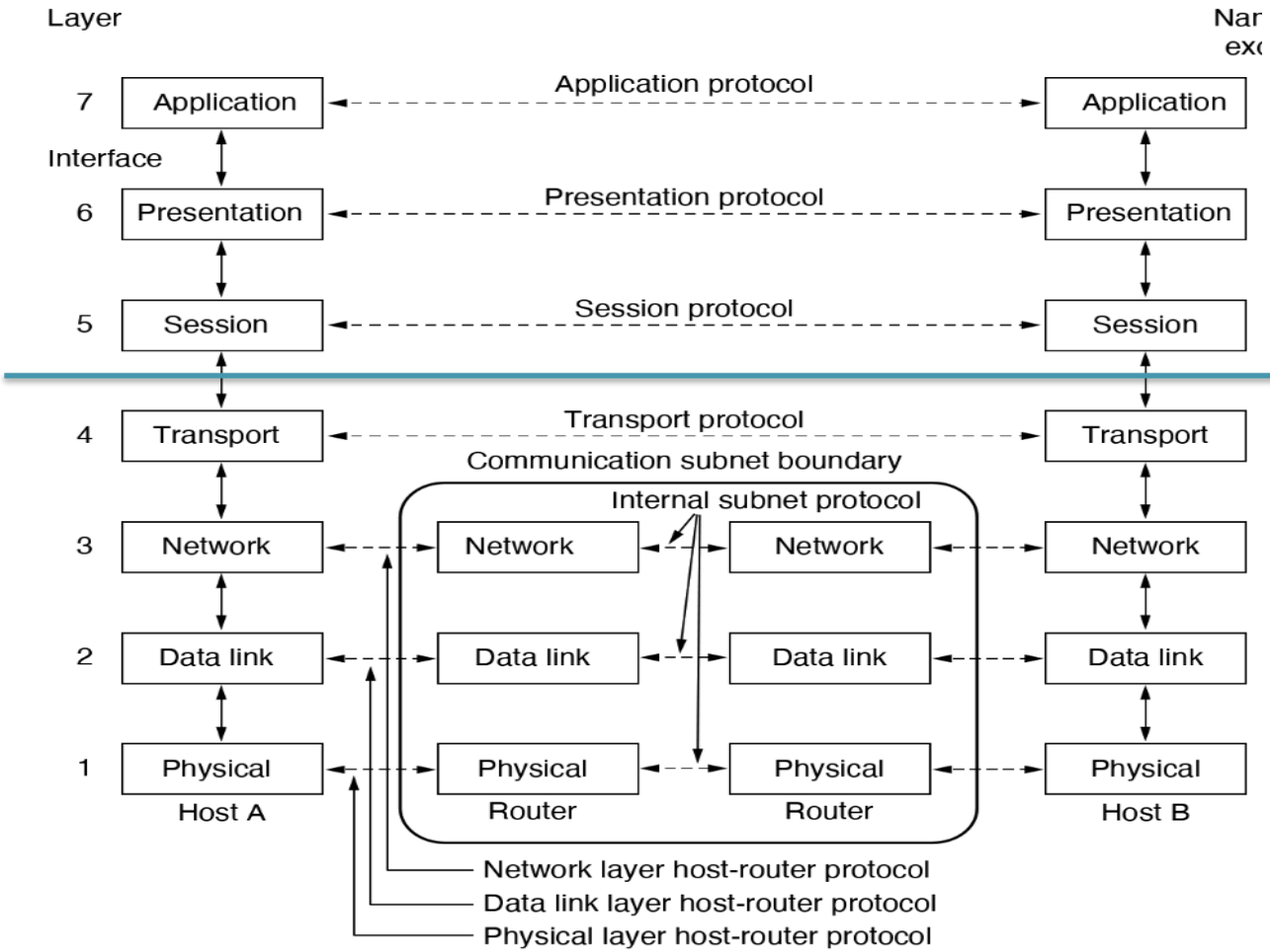
- A socket is how user-space code sends messages to kernel-space networking code
- A “Hello, world!” networking program will create a socket and then send a message.
- A process sends and receives through a socket

  the “doorway” leading in/out of the application



© 2025 University of Melbourne

First point of contact: sockets

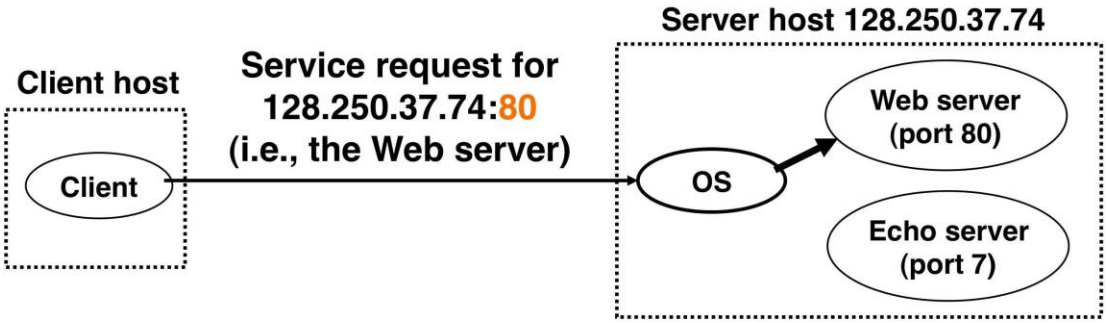


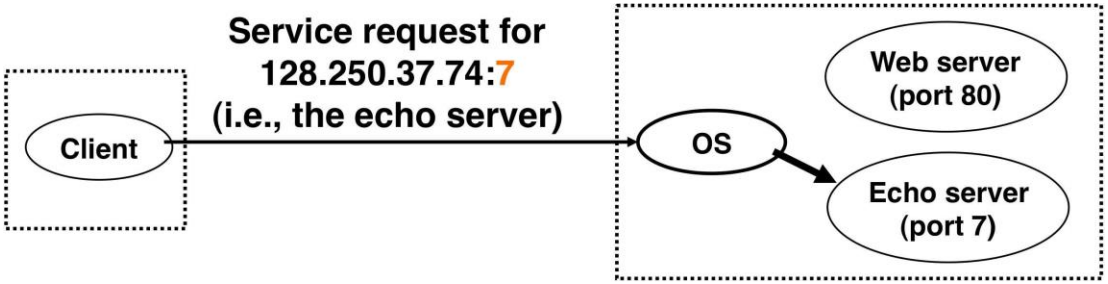
- Use data
- Tidy up e2e
- Get data e2e
- Tidy up p2p
- Get data p2p

Using sockets

Old (IPv4) IP addresses are four bytes separated by dots.

- The "address" of a socket is the 5-tuple:
  - Protocol
  - local-IP
  - local-port number
  - remote-IP
  - remote-port number.
- Often misnamed "source"/"dest" not "local"/"remote"
- We will study IP addresses later





© 2025 University of Melbourne

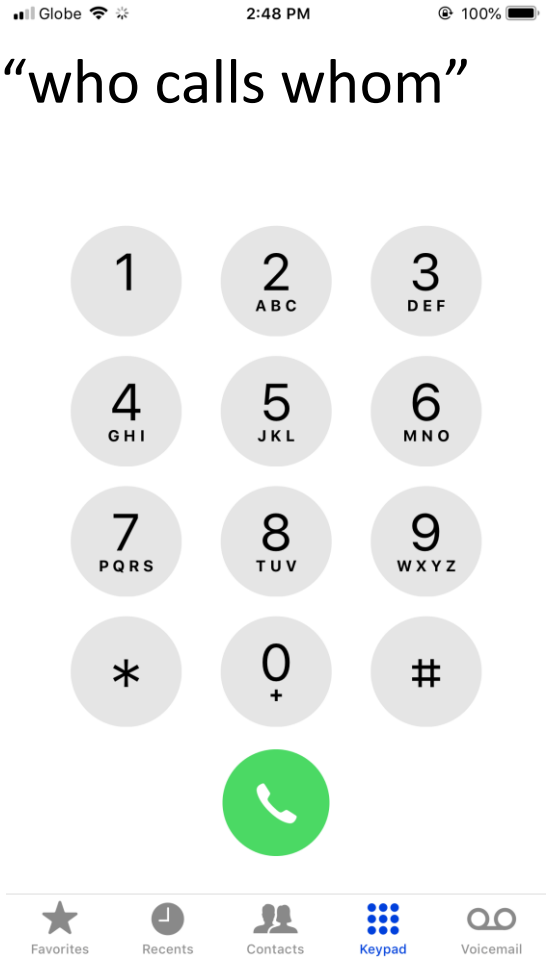
Berkeley Sockets

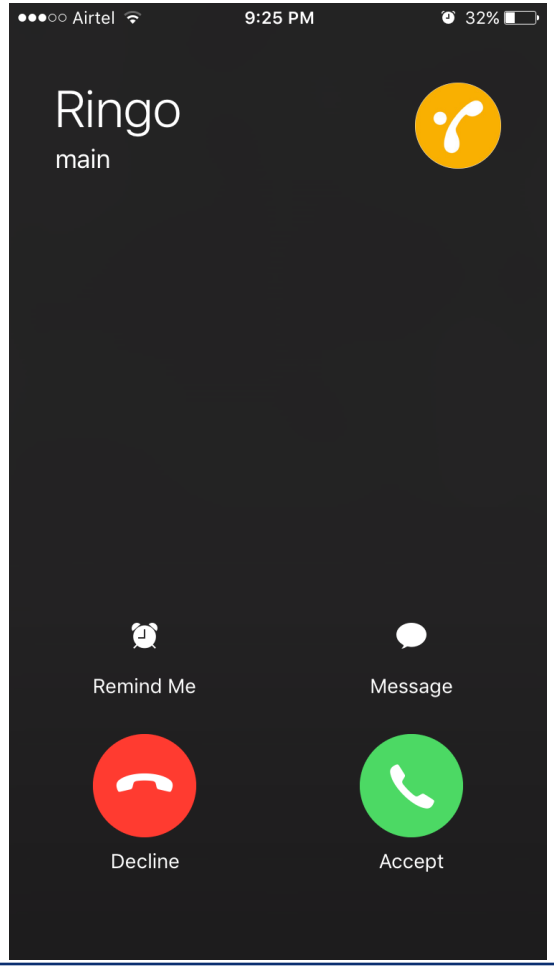
- Socket interface
  - All major OSes copy Berkeley UNIX
  - portability
- In UNIX, everything is like a file
  - all input is like reading a file
  - all output is like writing a file
  - file is “addressed” by an integer file descriptor
- API implemented as system calls:
  - examples include connect(), read(), write(), close()

© 2025 University of Melbourne

“Clients” vs “servers”

Just “who calls whom”





© 2025 University of Melbourne

“Clients” vs “servers”

Receiver

- Make sure SIM card is in
- Make sure phone is on
- Make sure phone is not in aeroplane mode

➢ Ready to accept a call from anyone

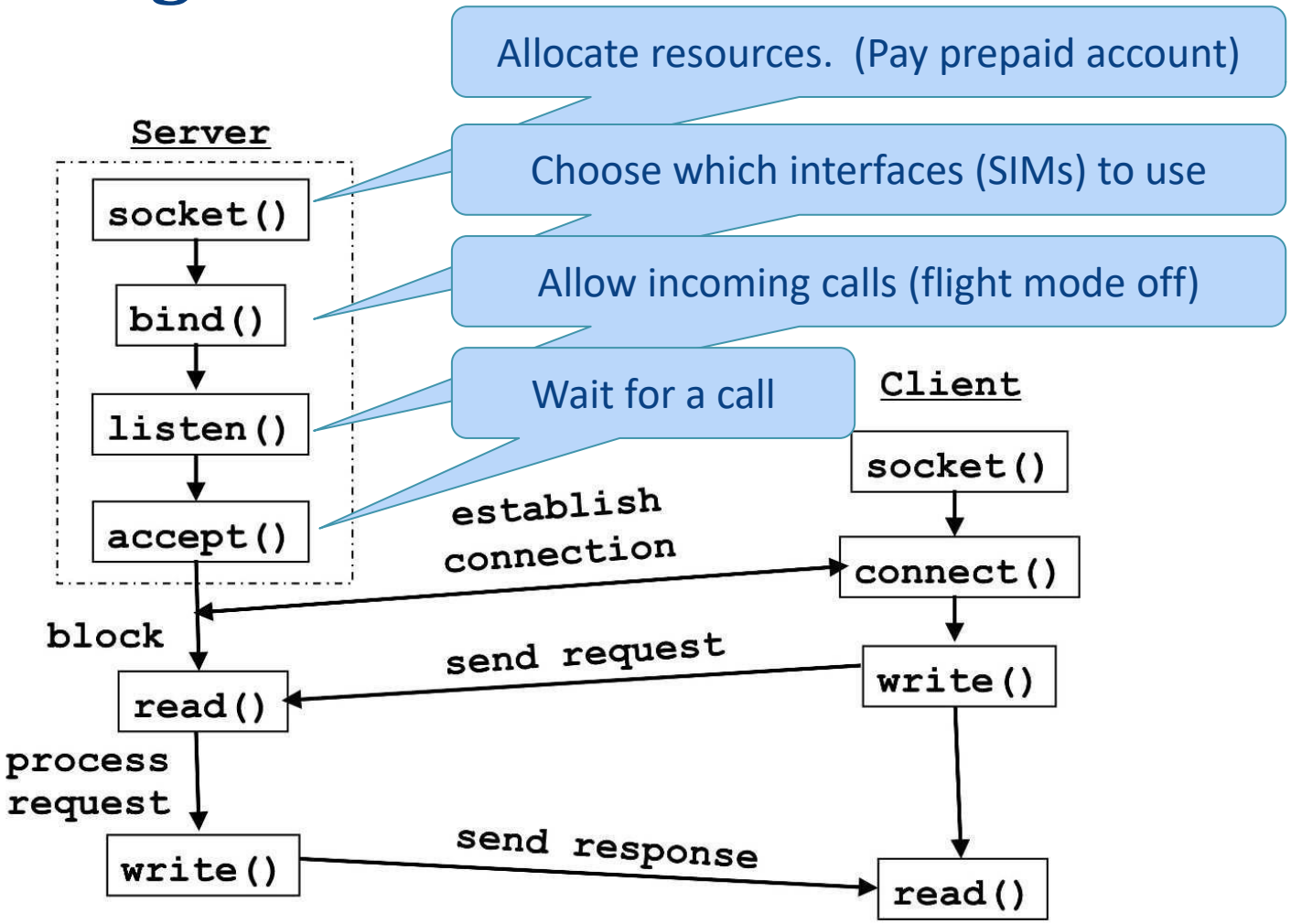
Caller

- As above
- Dial phone number to say *whom* you want to connect to

Afterwards: Both sides can talk and listen equally

© 2025 University of Melbourne

Using sockets

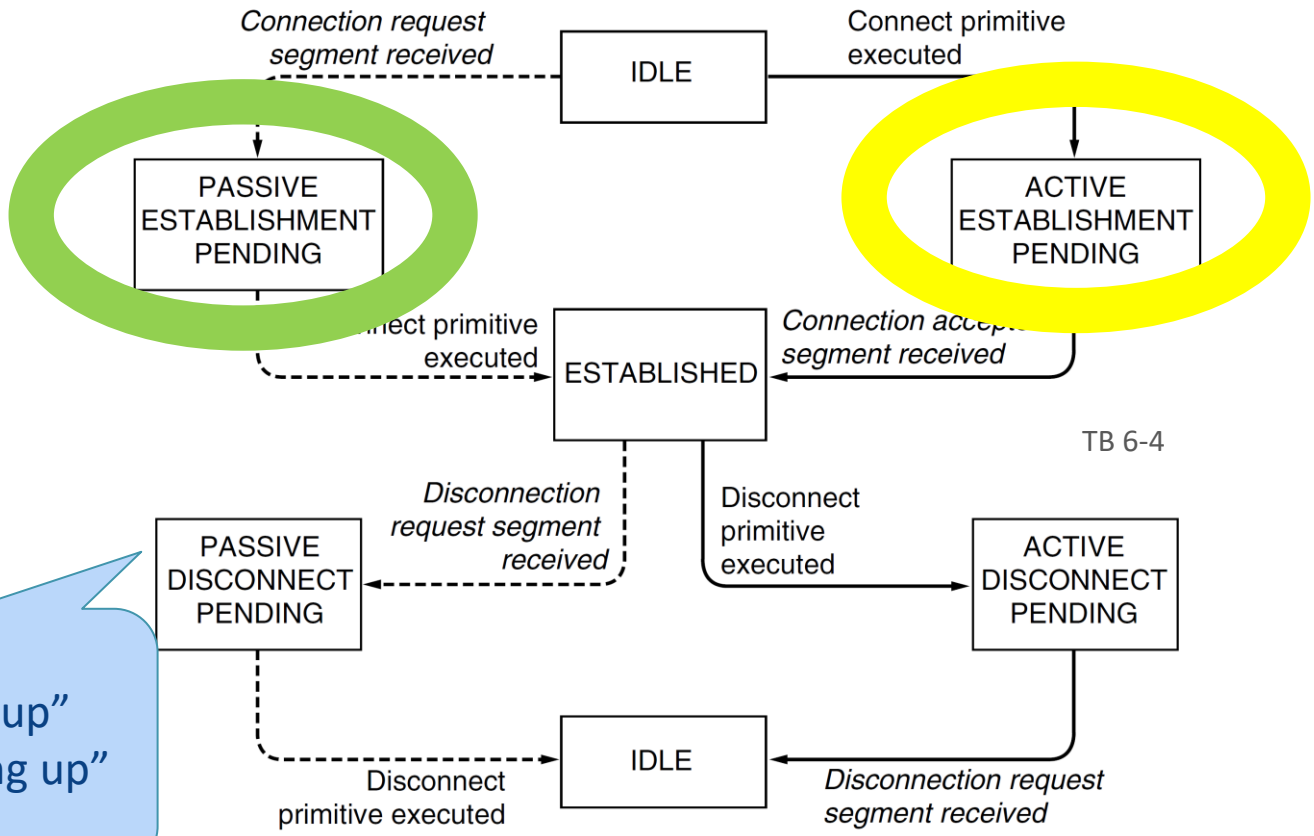


Socket Primitives

State	Description
SOCKET	Creates a new communication endpoint
BIND	Associate a local address with a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Passively establish an incoming connection (block until then)
CONNECT	Actively attempt to establish a connection
SEND	Send some data over a connection (write())
RECEIVE	Receive *some* data from the connection (read())
CLOSE	Release the connection

© 2025 University of Melbourne

Simplified (6-)State diagram for Connection Management



"You hang up"
"No, you hang up"

C Listening ("server") socket

```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int re = 1, s, n;
char recvBuff[1024];

struct sockaddr_in address;
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);


Int listenfd = socket (AF_INET,
                       SOCK_STREAM, 0);
```

```
setsockopt(listenfd, SOL_SOCKET,
                SO_REUSEADDR,
                &re, sizeof(re));
bind(listenfd, (struct sockaddr*)&address,
        sizeof(address));
// 10 is max num of client connections  to queue
listen(listenfd, 10);
struct sockaddr_storage client_addr;
socklen_t client_addr_size =sizeof client_addr;
int connfd = accept(listenfd,
                (struct sockaddr*)&client_addr,
                 &client_addr_size);
read (connfd, recvBuff, sizeof (recvBuff));
write (connfd, "Hello, back!\r\n");
close (connfd);
```

Block

Server has two sockets

```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int re = 1, s, n;
char recvBuff[1024];

struct sockaddr_in address;
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);


Int listenfd = socket (AF_INET,
                       SOCK_STREAM, 0);
```

```
setsockopt(listenfd, SOL_SOCKET,
           SO_REUSEADDR,
           &re, sizeof(re));
bind(listenfd, (struct sockaddr*)&address,
     sizeof(address));
// 10 is max num of client connections  to queue
listen(listenfd, 10);
struct sockaddr_storage client_addr;
socklen_t client_addr_size =sizeof client_addr;
int connfd = accept(listenfd,
          (struct sockaddr*)&client_addr,
           &client_addr_size);
read (connfd, recvBuff, sizeof (recvBuff));
write (connfd, “Hello, back!\r\n”);
close (connfd);
```

© 2025 University of Melbourne

Server has two sockets

- The server code has two sockets: listenfd, connfd
- The "listening socket" is a "half socket"
  - Protocol
  - Local IP address
  - Local port number
- It's role is like a receptionist (or automated "Press 1 for…")
  - Wait for calls
  - Put them through to another line (connfd)
- The connection socket is the one we can read/write
  - It is identified by a full 5-tuple

© 2025 University of Melbourne

Blocking and non-blocking reads

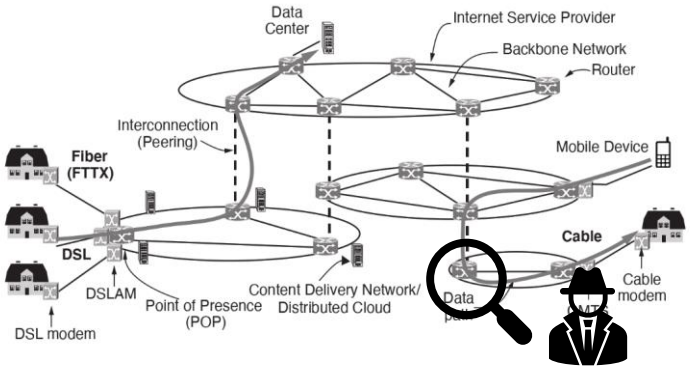
- Reading from a stored file: always data to read until EOF
- Over a network: data arrives in stages
  - Always check how much was read
  - What is read may be less than what was in a write() call
    (We'll see more detail when we discuss TCP vs UDP)

```
while ((n = read(connfd, recvBuff, sizeof(recvBuff)-1)) > 0) {
    // process received buffer
}
```

  - If the socket is *blocking* (see fcntl and O_NONBLOCK), it waits until there is *some* data
    - The above loop reads the whole connection
  - If *non-blocking*, this just reads data that has arrived
    - May return 0 bytes even if not at the end of the connection

© 2025 University of Melbourne

Transport layer security / Secure socket layer



- Sockets carry plain text
- TLS (formerly SSL) encrypts data
- Extra “handshake” at the beginning to verify and set up crypto

TLS: simple or sophisticated

- Simple to use TLS just for encryption
  - Many web sites require TLS, so clients need to be able to do this
- Also provide identity verification, which is harder
  - Needed when you need to trust the server
  - E.g., Anything where you provide information

© 2025 University of Melbourne

Insecure TLS

- Websites now need TLS
- *If* you don't need to verify the web site, you can use this OpenSSL code
- This is only to be used for insecure applications forced to use TLS.

```
// Initialise OpenSSL
SSL_library_init();
OpenSSL_add_all_algorithms();
// Create connection
SSL_CTX *ctx =
        SSL_CTX_new(SSLv23_client_method());
SSL *ssl = SSL_new(ctx);
BIO *bio = BIO_new_ssl_connect(ctx);
BIO_set_conn_hostname(bio, "localhost:993");
BIO_get_ssl(bio, &ssl);
SSL_set_mode(ssl ,
                SSL_MODE_AUTO_RETRY);

BIO_puts(bio, "Hello, world!");

BIO_free_all(bio);
SSL_CTX_free(ctx);
```

© 2025 University of Melbourne

TLS in Rust (Not examinable; for those using Rust in project 2)

```
extern crate rustls; // 0.17.0

use io::Read;
use io::Write;
use std::io;

fn main() {
   let mut socket = std::net::TcpStream::connect("www.google.com:443").unwrap();
   let mut config = rustls::ClientConfig::new();
   config
      .root_store
      .add_server_trust_anchors(&webpki_roots::TLS_SERVER_ROOTS);
   let arc = std::sync::Arc::new(config);
   let dns_name = webpki::DNSNameRef::try_from_ascii_str(“localhost").unwrap();
   let mut client = rustls::ClientSession::new(&arc, dns_name);
   let mut stream = rustls::Stream::new(&mut client, &mut socket); // Create stream
                                                   // Instead of writing to the client, you write to the stream
   stream
      .write(b“Hello, world!")
      .unwrap();
   let mut plaintext = Vec::new();
   stream.read_to_end(&mut plaintext).unwrap();
   io::stdout().write_all(&plaintext).unwrap();
}
```

(https://stackoverflow.com/questions/60357636/how-do-i-make-a-tls-connection-using-the-rustls-library)

© 2025 University of Melbourne

What now?

This just creates a data pipe between two hosts

What do we *do* with it?

```
setsockopt(listenfd, SOL_SOCKET,
                SO_REUSEADDR,
            &re, sizeof(re));
bind(listenfd, (struct sockaddr*)&address,
        sizeof(address));
// 10 is max num of client connections  to queue
listen(listenfd, 10);
struct sockaddr_storage client_addr;
socklen_t client_addr_size =sizeof client_addr;
int connfd = accept(listenfd,
            (struct sockaddr*)&client_addr,
             &client_addr_size);
read (connfd, recvBuff, sizeof (recvBuff));
write (connfd, “Hello, back!\r\n”);
close (connfd);
```

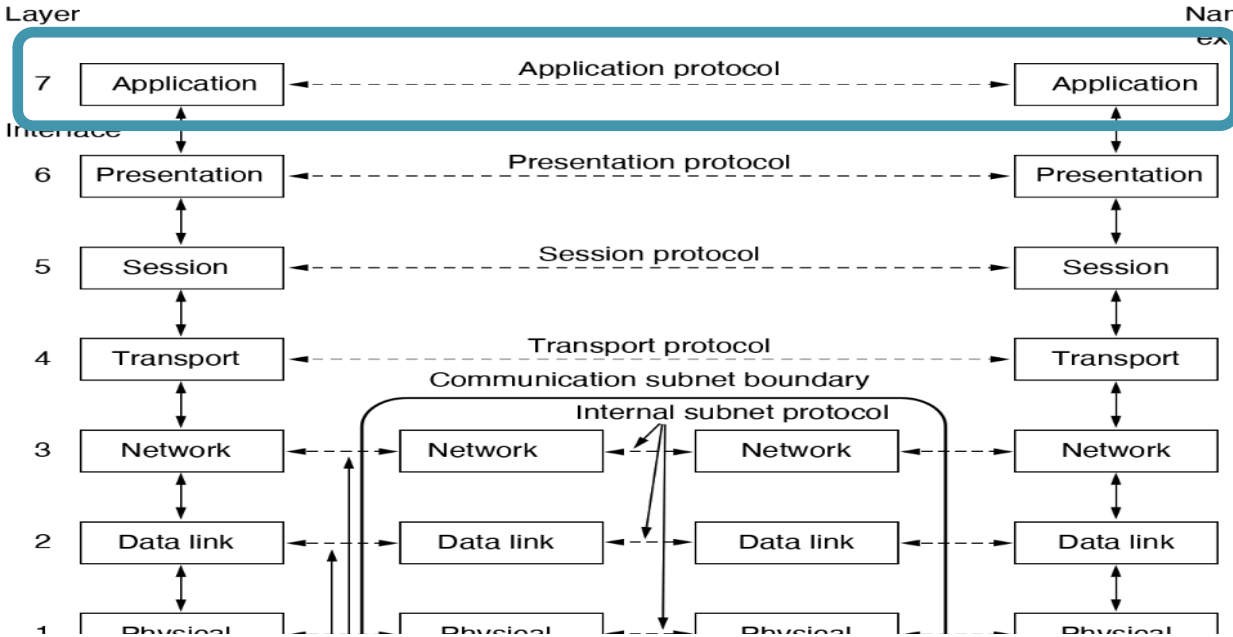
© 2025 University of Melbourne

What now?

This just creates a data pipe between two hosts

What do we *do* with it?

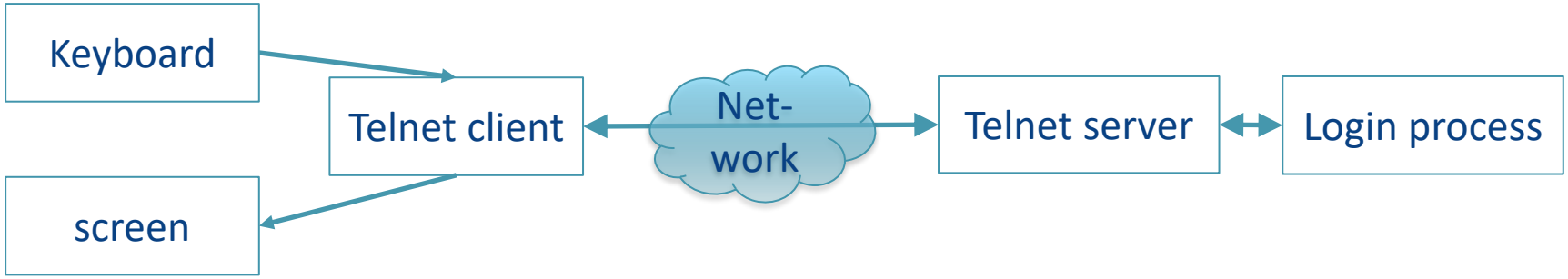
- Terminal emulation
- Email
- Web
- Virtual desktop
- Video streaming



The following old/draft slides are not assessable.
They are only left here in case someone finds them interesting.
Please let me know on Ed if you find them distracting and I will stop including them.

telnet – emulating a dumb terminal

- Telnet lets a host open a command-line session on another
- Client is trivial
  - (Non-blocking) fscanf keystrokes, and write them to the socket
  - Read characters from socket, and fprintf them locally
- Server is slightly more complex
  - Starts a local "login process", and feeds it with data from the socket
  - Network component is still trivial



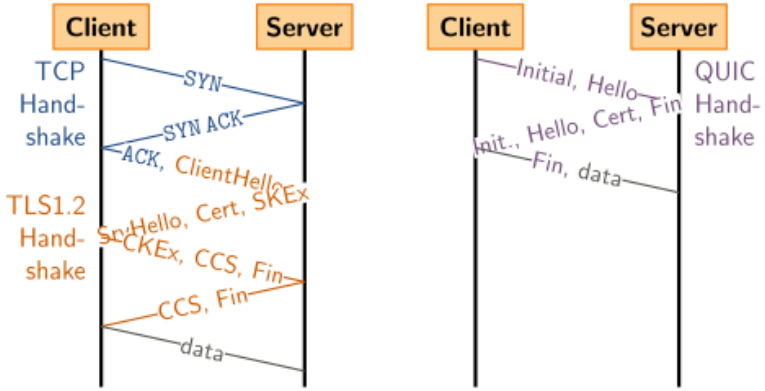
© 2025 University of Melbourne

QUIC

- QUIC is another reliable transport protocol
  - Originally for running HTTP over, but can be used for anything
- Standardized in 2021
- Runs over UDP
  - (firewalls block protocols other than TCP/UDP)
- Multiple byte streams instead of just one

© 2025 University of Melbourne

QUIC benefits



- Multiple byte streams, not just c
  - If a packet is lost, only one stream
- TLS (transport layer security) handshake is combined with TCP handshake
  - fewer idle round trip times (RTTs).
- Can reconnect to a different IP address mid flow
  - Useful if switching from WLAN to 4G or ethernet

© 2025 University of Melbourne

QUIC sockets

Three types of sockets to expose the QUIC API

- **Listener sockets:** like regular TCP/UDP listeners, they bind to a port and are used to accept incoming connections. Listeners can be closed when the application no longer desires to accept new connections.
- **Connection sockets:** these sockets are created by connect or accept calls. Unlike TCP or UDP sockets, however, these sockets cannot be used to send or receive data and are only used to manage the streams belonging to the connection.
- **Stream sockets:** these are the sockets that provide send and receive functions so peers can exchange data. They can be created either by accepting on a connection socket (for streams opened by the peer), or by calling connect on the connection socket to create a new, locally initiated stream. Streams can be closed by either peer at any point during the connection.

[https://blogs.cisco.com/cloud/building-fast-quic-sockets-in-vpp]

© 2025 University of Melbourne

Acknowledgement

- The slides are based on slides prepared by Chris Culnane based on material developed previously by: Michael Kirley, Zoltan Somogyi, Rao Kotagiri, James Bailey and Chris Leckie.
- Some of the images included in the notes were supplied as part of the teaching resources accompanying the text books listed in lecture 1.
  - (And also) Computer Networks, 6th Edition, Tanenbaum A., Wetherall. D. https://ebookcentral.proquest.com/lib/unimelb/detail.action?docID=6481879
- Textbook Reference: 3.5.4, 3.5.5, 3.4, bits of 3.2. The text doesn't cover sockets in C, but many books do; Google's first pick is: TCP/IP Sockets in C, by Donahoo. M, Calvert. K

© 2025 University of Melbourne

Sockets in C

- Headers

```
#define _POSIX_C_SOURCE 200112L // Required for VSCode
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>
```

- Variables

```
int listenfd = 0, connfd = 0, re = 1, s, n;
char sendBuff[1024];
struct addrinfo hints, *res, *rp;
```

- Create a socket

```
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE;
s = getaddrinfo(NULL, "5000", &hints, &res);
listenfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &re, sizeof(re));
```

AF_INET6

© 2025 University of Melbourne

Create IPv6 socket

- getaddrinfo() can return multiple addresses as a linked list
- If you want the IPv6 address in particular, you may have to loop over the responses:

```
for (p = res; p!= NULL; p = p->ai_next) {
   if (p->ai_family == AF_INET6
          && (sock = socket(p->ai_family,
                            p->ai_socktype,
                            p->ai_protocol)) < 0 )
   {
      // socket creation was attempted but failed
   }
}
// Use  sock  here if it is >= 0.
```

© 2025 University of Melbourne

Sockets in C - server

- Bind and listen

p of loop

```
bind(listenfd, res->ai_addr, res->ai_addrlen);
// maximum number of client connections to queue
listen(listenfd, 10);
```

Put TCP state machine in LISTEN state

- Accept, write/send, close

```
struct sockaddr_storage client_addr;
socklen_t client_addr_size = sizeof client_addr;
connfd = accept(listenfd, (struct sockaddr*)&client_addr, &client_addr_size);
snprintf(sendBuff, sizeof(sendBuff), "Hello World!\n");
n = write(connfd, sendBuff, strlen(sendBuff));
close(connfd);
```

Block

- Wait until one of several files is ready to read / write
  - select(), pselect(), poll()

© 2025 University of Melbourne